Dart Programlama Diline Giriş

Temel Kavramlar, Değişkenler, Operatörler ve Kod Dokümantasyonu

Eksiksiz Ders ve Sunum Rehberi • Dart SDK & Sound Null Safety

DART PROGRAMLAMA DİLİ

Ders İçeriği Dört Ana Modülde Kapsamlı Olarak Ele Alınmaktadır

01

Dart'a Giriş

Main fonksiyonu, tür güvenliği, kontrol akışı, fonksiyonlar ve OOP temelleri.

02

Değişkenler

Var ile tür çıkarımı, açık türler, Null Safety, late, final ve const kavramları.

03

Operatörler

Aritmetik, karşılaştırma, mantıksal, null-aware, cascade ve spread operatörleri.

04

Yorumlar

Tek/çok satırlı yorumlar, dokümantasyon yorumları (///) ve temiz kod ilkeleri.

Tüm Dart Programları Giriş Noktası Olan main() Fonksiyonu ile Başlar

Program Başlangıç Yapısı

• Giriş Noktası: Her Dart uygulamasının çalışmaya başladığı ana merkez main() fonksiyonudur.

• Dönüş Türü (void): void anahtar kelimesi, fonksiyonun geriye bir değer döndürmediğini belirtir.

• Konsol Çıktısı: print() fonksiyonu metin veya değişken değerlerini konsola yazdırmak için kullanılır.

• Sözdizimi (Syntax): Kod blokları süslü parantezler { } ile sınırlandırılır ve ifadeler noktalı virgül (;) ile biter.

```
// En temel Dart programı

      void main() {
  print('Hello, World!');

// Değişken kullanımı örneği
     var name = 'Dart';
 print('Merhaba, $name!');
              }
```

Dart Tür Güvenliğini Otomatik Tür Çıkarımı (Type Inference) ile Birleştirir

int & double

Sayısal Veri Türleri

int tam sayıları (1977), double ise ondalıklı sayıları (3.14) temsil eder.

String & bool

Metin ve Mantık

String metinsel ifadeleri, bool ise doğru/yanlış (true/false) değerlerini tutar.

List & Map

Koleksiyon Yapıları

List sıralı eleman listelerini, Map ise anahtar-değer (key-value) çiftlerini yönetir.

var Anahtarı

Tür Çıkarımı

var ile tanımlanan değişkenin türü, atanan ilk değerden otomatik olarak tespit edilir.

Koşullu İfadeler ve Döngüler Kodun Çalışma Mantığını Yönlendirir

```
if (score >= 50) {
print('Tebrikler, geçtiniz!');
} else {
print('Tekrar deneyiniz.');
}
```

```
// Standart for döngüsü
for (int i = 0; i < 3; i++) {
print('Sayı: $i');
}

// Liste üzerinde for-in
var list = ['A', 'B'];
for (var x in list) print(x);
```

Fonksiyonlar Kod Tekrarını Önler; Ok Gösterimi Tek İfadeleri Kısaltır

Standart Fonksiyon Yapısı

Parametre alan, hesaplama yapan ve return anahtarı ile değer döndüren klasik yapı.

```
int karesi(int sayi) {
  return sayi * sayi;
}

void main() {
print(karesi(5)); // 25
}
```

Ok Gösterimi / Arrow Syntax (=>)

Tek bir süslü parantez ve return ifadesi içeren fonksiyonları kısa yazmak için kullanılır.

```
// Tek satırlı ok fonksiyonu
String selamla(String isim) =>
    'Merhaba, $isim!';

bool ciftMi(int n) => n % 2 == 0;
```

Sınıflar Veri ve Davranışları Alanlar, Metotlar ve Yapıcılar ile Paketler

Alanlar (Fields)

Nesnenin durumunu ve verilerini tutan değişkenlerdir.

Metotlar (Methods)

Sınıfın gerçekleştirebildiği işlev ve davranışlardır.

Yapıcılar (Constructors)

Nesne ilk oluşturulurken başlangıç değerlerini veren özel fonksiyonlardır.

```
class Spacecraft {
    String name;
DateTime? launchDate;

// Constructor (Yapıcı Metot)
Spacecraft(this.name, this.launchDate);

// Metot
void describe() {
print('Uzay Aracı: $name');
if (launchDate != null) {
print('Fırlatma: ${launchDate!.year}');
}
}
}
```

Kalıtım, Mixin, Kütüphaneler ve Asenkron Yapılar Esnek Mimariler Sunar

enum

Sabit Değer Kümeleri

Sınırlı ve belirli sayıda sabit değeri tanımlamak için kullanılır.

extends & with

Kalıtım ve Mixin

Kalıtım (extends) ile sınıf genişletilir; mixin (with) ile kod tekrarı olmadan davranış eklenir.

import & try/catch

Kütüphane ve Hata

dart:math gibi kütüphaneler içe aktarılır; try-catch ile istisnalar (exceptions) yönetilir.

Future & async/await

Asenkron Programlama

Uzun süren işlemleri (ağ isteği, dosya okuma) ana iş parçacığını engellemeden yürütür.

Değişkenler Tür Çıkarımı veya Açık Tür Tanımı ile Değer Depolar

```
var name = 'Bob'; // String çıkarımı
var year = 1977;  // int çıkarımı
var antenna = ['A', 'B']; //
        List<String>
```

```
String name = 'Bob';
  int year = 1977;
double temp = 21.5;
bool isReady = true;
```

Null Güvenliği Çalışma Zamanı Hatalarını Derleme Zamanında Önler

Null Olmayan Türler (Non-Nullable)

Varsayılan olarak Dart değişkenleri null değer alamaz. Bu sayede NullPointerException / NoSuchMethodError hataları tamamen engellenir.

```
String title = 'Dart';
// title = null; // DERLEME HATASI!

int age = 25; // Null atanamaz
```

Null Olabilen Türler (Nullable ?)

Bir değişkenin null değer alabileceği söylenmek isteniyorsa tür adının sonuna soru işareti (?) eklenir.

```
String? name; // Varsayılan değer null
name = 'Ahmet';
name = null; // Geçerli işlem!
```

late Anahtar Kelimesi Değişken Başlatmasını Sonraya Erteler

late Kullanım Amaçları

```
late String description;

void main() {
  // Değişken burada başlatılıyor
  description = 'Dart programlama dili';
  print(description);
}

// Tembel yükleme örneği
late String data = _heavyCalculation();
```

final Çalışma Zamanında, const İse Derleme Zamanında Sabitlenir

```
final name = 'Bob';
final currentYear =
    DateTime.now().year;
// currentYear çalışma zamanında
    belirlenir.
```

```
const pi = 3.14159;
const atmosphere = 1.01325;
// const DateTime.now(); // DERLEME
    HATASI!
```

MODÜL 2: DEĞİŞKENLER

Doğru Değişken Türünü Seçmek Kod Okunabilirliğini ve Güvenliğini Yükseltir

1. Sabitlik Önceliği — Değeri değişmeyecek tüm değişkenlerde öncelikle 'final' veya 'const' kullanın.

2. Derleme Sabitleri — Derleme anında bilinen matematiksel ve yapısal değerler için 'const' tercih edin.

3. Açık Tür Beyanı — Türün karmaşıklaştığı veya okunabilirliğin önemli olduğu yerlerde açık tür yazın.

4. Null Yönetimi — Gereksiz yere nullable (?) tür kullanmaktan kaçının; null olasılığını explicit yönetin.

5. Anlamlı İsimlendirme — Değişken isimlerinde lowerCamelCase kuralına ve açıklayıcı kelimelere uyun.

MODÜL 3: OPERATÖRLER

Aritmetik Operatörler Sayısal Hesaplamaları ve Bölme İşlemlerini Yönetir

`+ , - , * , /`

Temel İşlemler

Toplama, çıkarma, çarpma ve standart bölme (sonuç double döner).

`~/ (Tam Bölme)`

İnteger Division

Bölüm sonucunun yalnızca tam sayı kısmını alır (örn. 7 ~/ 2 = 3).

`% (Mod)`

Kalan Operatörü

İki sayının bölünmesiyle elde edilen kalanı hesaplar (örn. 7 % 2 = 1).

`++ / --`

Artırma / Azaltma

Sayısal değişkenin değerini 1 artırır veya 1 azaltır (prefix/postfix).

MODÜL 3: OPERATÖRLER

Karşılaştırma ve Mantıksal Operatörler Koşul Kontrollerini Sağlar

Karşılaştırma

Eşitlik ve Büyüklük

- == (Eşit mi)
- != (Eşit değil mi)
- > , < (Büyük/Küçük)
- >= , <= (Büyük-Eşit/Küçük-Eşit)

Tür Testi

Tip Doğrulama

- is (Belirtilen türde mi)
- is! (Belirtilen türde değil mi)
- as (Tür dönüştürme/casting)

Mantıksal

Mantık Kapıları

- ! (Mantıksal DEĞİL)
- && (Mantıksal VE)
- || (Mantıksal VEYA)

Null Operatörleri Güvenli Değer Ataması ve Nesne Erişimi Sağlar

Üçlü Koşul (?:) [expr ? expr1 : expr2]

Koşul doğru ise expr1, yanlış ise expr2 değerlendirilir.

Null Birleştirme (??) [expr1 ?? expr2]

expr1 null değilse onu, null ise varsayılan expr2 değerini döndürür.

Null-Aware Erişim (?.) [object?.property]

Nesne null ise hata vermek yerine güvenle null döndürür.

Cascade ve Spread Operatörleri Kod Yazımını Kısaltır ve Esnetir

```
// Cascade ile zincirleme erişim
var sb = StringBuffer()
  ..write('Merhaba ')
  ..write('Dart!');

print(sb.toString());
```

```
var list1 = [1, 2];
var list2 = [0, ...list1, 3]; // [0, 1, 2, 3]

List<int>? list3;
var list4 = [0, ...?list3]; // Null-safe spread
```

MODÜL 4: YORUMLAR

Dart Üç Farklı Yorum Türü ile Kod İçi Dokümantasyonu Destekler

// Tek Satırlı Yorum

Kod İçi Kısa Notlar

Tek bir satırı veya kod yanını açıklamak için kullanılır. Derleyici tarafından tamamen göz ardı edilir.

/* ... */ Çok Satırlı

Blok Açıklamalar

Birden fazla satıra yayılan detaylı açıklamalardır. Dart'ta çok satırlı yorumlar iç içe (nested) geçebilir.

/// Dokümantasyon

dartdoc Belgelendirmesi

Sınıf, metot ve API dokümantasyonu üretmek için kullanılır. IDE gösterimlerinde ve HTML dokümanda yer alır.

Yorumlar Kodun Ne Yaptığını Değil Neden Yapıldığını Açıklamalıdır

Neden Sorusuna Yanıt Verin

Kod zaten 'ne' yapıldığını gösterir; yorumlar kararların arkasındaki 'neden' mantığını sunmalıdır.

Yorumları Güncel Tutun

Kod değiştiğinde yenilenmeyen yorumlar yanlış yönlendirir ve hatalara sebep olur.

Dokümantasyon Yorumları Kullanın

Kamusal (public) sınıf ve fonksiyonların başında /// dokümantasyon yorumu yazın.

Gereksiz Yorumlardan Kaçının

Aşırı belirgin veya kendini tekrar eden ('// i'yi 1 artır') yorumları temizleyin.

ÖZET VE DEĞERLENDİRME

Dart Temelleri: Güvenli Tür Yapısı, Zengin Operatörler ve Temiz Dokümantasyon

Dart Syntax & OOP

main() giriş noktası, güçlü tür çıkarımı (var) ve sınıflarla nesne yönelimli mimari.

Değişkenler & Null Safety

Sound Null Safety ile çalışma zamanı hatalarından korunma, late, final ve const ayırımı.

Operatörler & İfadeler

Aritmetik, mantıksal, null-aware (??, ?.), cascade (..) ve spread (...) operatörleri.

Dokümantasyon

/// dokümantasyon yorumları ve okunan, bakımı kolay kod yazma standartları.